Agentic Returns: Codebase & Reinforcement Learning Architecture

Technical Architecture & Concept Document

1. Executive Summary

The Agentic Returns system represents a paradigm shift from static, rules-based customer service forms to an autonomous, multi-agent orchestration layer.

Instead of relying on human agents to manually look up databases, read policies, and navigate decision trees, this codebase models the return process as a **Markov Decision Process (MDP)** within a **Reinforcement Learning (RL)** framework. The primary goal is to reach a terminal state (a confident approval or rejection of the return) while incurring the lowest possible "cost" (human intervention and friction).

This document breaks down the codebase conceptually, highlighting the specific sections of the Python backend that drive this Actor-Critic RL architecture.

2. The RL Paradigm: Markov Decision Process (MDP)

In RL terms, processing a return is a step-by-step game.

- **State (S_t):** The known facts about the transaction (Item details, purchase date, customer history).
- **Action (A_t):** The AI deducing answers to 5 Critical Questions required for a return.
- **Reward (R_t):** Positive reinforcement (+1) for answers that comply with official store policy, and negative penalties (-1) for policy deviations or missing data that require human follow-up.

3. Phase 1: State Space Initialization (S_t)

Before the AI makes any decisions, the system must gather deterministic data. If an LLM is forced to guess dates or prices, it will hallucinate. We solve this by grounding the State Space in absolute reality using Snowflake database lookups and deterministic Python math.

Important Code Highlight: Robust Database Extraction

In `store_sales_lookup.py`, we execute a query joining the `STORE_SALES` and `DATE_DIM` tables. Because different database drivers handle casing differently (uppercase vs. lowercase keys), we use a helper function `_get_val` to safely extract the data.

```
# store_sales_lookup.py
def _get_val(row: dict, key: str) -> Any:
    """Helper to safely extract dictionary values regardless of driver case-sensi
    if not row:
        return None
```

```
    for k, v in row.items():
        if k.lower() == key.lower():
            return v
    return None
```

Important Code Highlight: Deterministic Math Bypass

In `researcher_agent.py` , instead of passing the raw purchase date to the LLM and asking it to do 90-day math (which LLMs struggle with), we use native Python. This hardcodes a perfect State for Question 3.

```
# researcher_agent.py (Inside investigate_return)
elif q_spec["key"] == "purchase_date" and sales_lookup.get("valid"):
    # ...
    # 🔥 Native deterministic math for Purchase Date (Bypass LLM)
    p_date = datetime.fromisoformat(raw_date.split("T")[0]) if "T" in raw_date el
    days_since = (datetime.now() - p_date).days
    is_valid = days_since <= 90

    validation_result = {
        "valid": is_valid,
        "compliance_status": "compliant" if is_valid else "deviation",
        "exact_issue": f"Standard Return Policy (90-day): Purchase was {days_sinc
        # ...
    }
    skip_policy_agent = True # Completely bypass the AI critic for perfect math
```

4. Phase 2: The Actor-Critic Loop

Once the State is initialized, the system enters the core RL loop using a dual-agent setup.

The Actor: Researcher Agent

The **Researcher Agent** acts as the RL Actor. It observes the State (customer remarks, item details) and takes an **Action**: proposing a hypothesized answer to one of the 5 Critical Questions.

```
# researcher_agent.py
async def _formulate_answer_llm(self, q_spec: dict, context: dict[str, Any]) -> c
    # ... (Builds prompt with resolved context) ...
    response = litellm.completion(
        model=LLM_MODEL,
        messages=[{"role": "system", "content": RESEARCHER_SYSTEM_PROMPT}, {"role
        temperature=0.2, # Low temperature for logical deduction
        max_tokens=300,
    )
    # The Actor returns its hypothesis and self-reported confidence
```

The Critic: Policy Agent (RAG)

The Actor is not allowed to make the final decision. It submits its hypothesis to the **Policy Agent**, which acts as the Critic. The Critic queries the actual Walmart policies via Retrieval-Augmented Generation (RAG) and compares the Actor's hypothesis against the rules.

The Critic then issues the **Reward Signal** via the `_adjust_answer` method.

```
# researcher_agent.py
def _adjust_answer(self, initial_answer: dict[str, Any], validation_result: dict[
    """
    Adjust answer based on Policy Agent validation feedback.
    This acts as the Reward Function.
    """
    if validation_result["valid"]:
        # Positive Reward: Boost confidence
        return (initial_answer["answer"], min(1.0, initial_answer["confidence"] +
    else:
        # Negative Penalty: Slash confidence due to deviation
        return (initial_answer["answer"], initial_answer["confidence"] * 0.8)
```

5. Phase 3: Dewey Framework & Optimization (Policy Gradients)

When the State Space is incomplete (e.g., the customer didn't mention their return reason), the system cannot proceed. It flags a missing question and pauses for a Human-in-the-Loop follow-up.

In an RL model, human intervention is tracked as a severe negative penalty.

Important Code Highlight: Identifying Gaps

```
# researcher_agent.py (Inside _analyze_customer_remarks)
else:
    analysis["missing_questions"].append({
        "question_id": q_spec["id"],
        "question": q_spec["question"],
        "needed": True,
        "reason": "Customer remarks did not provide enough information."
    })
    # This triggers the UI to ask the human agent for help
```

The Dewey Framework Enhancement

To prevent recurring penalties, the system leverages the **Dewey Framework Enhancement**. When a penalty occurs, Reinforcement Learning empowers **Tutorial Agents** to step in. These agents learn optimal scaffolding strategies through student (agent) interactions. By utilizing **Policy Gradients**, the system actively optimizes the question sequences and difficulty progression based on previous feedback.

Over time, the model learns the most efficient sequence of context-gathering to minimize the probability of triggering that "missing data" penalty, creating a self-optimizing customer service workflow.

6. Conclusion

By structuring the `ResearcherAgent` as a dynamic orchestration layer rather than a static script, the codebase achieves true agentic behavior. The separation of concerns between the deterministic database lookups (State), the LLM reasoning (Actor), and the strict RAG validation (Critic) ensures that the system remains both highly autonomous and strictly compliant with store policies.